

Question description

There are n boys and m girls in a school. Next week a school dance will be organized. A dancing pair consists of a boy and a girl, and there are k potential pairs.

Your task is to find out the maximum number of dance pairs and show how this number can be achieved.

Constraints

- $1 \leq n, m \leq 500$
- $1 \leq k \leq 1000$
- $1 \leq a \leq n$
- $1 \leq b \leq m$

Input

The first input line has three integers n , m and k : the number of boys, girls, and potential pairs. The boys are numbered $1, 2, \dots, n$, and the girls are numbered $1, 2, \dots, m$.

After this, there are k lines describing the potential pairs. Each line has two integers a and b : boy a and girl b are willing to dance together.

Output

First print one integer r : the maximum number of dance pairs. After this, print lines describing the pairs. You can print any valid solution.

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

3 2 4
1 1
1 2
2 1
3 1

EXPECTED OUTPUT

2
1 2
2 1

Test Case 2

INPUT (STDIN)

3 2 4
1 1
1 2
2 3
3 2

EXPECTED OUTPUT

3
1 1
2 3
3 2

▼ Mandatory Test Cases

Test Case 1

KEYWORD

void link(int u,int v)

Test Case 2

KEYWORD

int bfs(int n)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

2

Test Case 2

TOKEN COUNT

600

Test Case 3

NLOC

84

Graph6.c +

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 #define MAXN 505
6 #define MAXM 1005
7
8 int head[MAXN], to[MAXM], nxt[MAXM], edge_cnt;
9 int match[MAXN], rmatch[MAXN];
10 int dist[MAXN];
11 int visited[MAXN];
12
13 void link(int u, int v) {
14     to[++edge_cnt] = v;
15     nxt[edge_cnt] = head[u];
16     head[u] = edge_cnt;
17 }
18
19 int bfs(int n) {
20     int q[MAXN], front = 0, rear = 0;
21     for (int i = 1; i <= n; i++) {
22         if (match[i] == 0) {
23             dist[i] = 0;
24             q[rear++] = i;
25         } else {
```

```

26         dist[i] = -1;
27     }
28 }
29 int reached_free = 0;
30 while (front < rear) {
31     int u = q[front++];
32     for (int e = head[u]; e; e = nxt[e]) {
33         int v = to[e];
34         int next_u = rmatch[v];
35         if (next_u == 0) {
36             reached_free = 1;
37         } else if (dist[next_u] == -1) {
38             dist[next_u] = dist[u] + 1;
39             q[rear++] = next_u;
40         }
41     }
42 }

```

```
43     return reached_free;
44 }
45
```

main | CH.SC.U4CSE25225 | Graph6.c

Ln 90, Col 1 Spaces: 4 UTF-8 CRLF C

```
47     for (int e = head[u]; e; e = nxt[e]) {
48         int v = to[e];
49         int next_u = rmatch[v];
50         if (next_u == 0 || (dist[next_u] == dist[u] + 1 && dfs(next_u))) {
51             match[u] = v;
52             rmatch[v] = u;
53             return 1;
54         }
55     }
56     dist[u] = -1;
57     return 0;
58 }
```

```
59
60int main() {
61    int n, m, k;
62    if (scanf("%d %d %d", &n, &m, &k) != 3) return 0;
63
64    for (int i = 0; i < k; i++) {
65        int u, v;
66        scanf("%d %d", &u, &v);
67        link(u, v);
68    }
69
70    int matching = 0;
71    while (bfs(n)) {
72        for (int i = 1; i <= n; i++) {
73            if (match[i] == 0) {
74                if (dfs(i)) {
```

```
75         matching++;
76     }
77 }
78 }
79 }
80
81 printf("%d\n", matching);
82 for (int i = 1; i <= n; i++) {
83     if (match[i] != 0) {
84         printf("%d %d\n", i, match[i]);
85     }
86 }
87
88 return 0;
89}
90
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

3 2 4

1 1

1 2

2 1

3 1

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Graph6.exe

2

1 2

2 1

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

3 2 4

1 1

1 2

2 3

3 2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Graph6.exe

3

1 1

2 3

3 2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>